English–Hindi Cross-Lingual Academic RAG Experimentation Platform

This is a research experimentation platform, not simply a chatbot. It will be used to study how Hindi–English language bridging affects retrieval quality, answer quality, faithfulness and computational cost in academic RAG. The implementation must therefore be reproducible, configurable and able to save all intermediate evidence.

Research question: When an academic question is asked in Hindi but the knowledge base is primarily in English, how does the method used to bridge the language gap affect retrieval quality, answer correctness, faithfulness and computational cost?

Four systems

ID	System	Pipeline	Final output
A	Direct LLM baseline	Question → LLM	Answer
B	English RAG	English question → English embedding → FAISS → English corpus → LLM	English answer
C	Translation-mediated Hindi RAG	Hindi → Hindi-to-English translation → English embedding → FAISS → English corpus → LLM	Hindi answer
D	Direct multilingual Hindi RAG	Hindi → multilingual embedding → FAISS → English corpus → LLM	Hindi answer

Primary comparison: C vs D. B is the English RAG baseline. A is the non-RAG baseline.

Detailed pipeline requirements

A — Direct LLM

- No retrieval.
- Prefer generation model as B/C/D.
- Save question, answer, model, generation settings and latency.

B — English RAG

- English question → English embedding → FAISS → top-k English chunks → LLM.
- Save chunk ID, document ID, page, rank, score/distance and retrieved text/reference.
- Return source references with the answer.

C — Translation-mediated Hindi RAG

- Save the original Hindi question unchanged.
- Translate Hindi → English and save the exact translated query.
- Use the same English corpus/index used by B.
- Retrieve top-k English chunks and save rankings/scores.
- Generate answer using the common generation configuration.
- Produce Hindi final answer; save any intermediate English answer separately.
- Save translation model/service and translation latency.

D — Direct multilingual Hindi RAG

- Do not translate the Hindi query before retrieval.
- Use a multilingual embedding model capable of Hindi↔English cross-lingual retrieval.
- Use the same English corpus as C.
- Save rankings, scores, sources, answer and latency.

Academic corpus

Initial subjects: Algorithms, Automata / Theory of Computation, DBMS, Operating Systems, Computer Networks. Target approximately 15–20 public/licensed academic documents.

Record:

- document_id
- title
- subject
- source type
- source URL/provenance where applicable
- language
- year if available
- file name
- page numbers

Prefer stable academic PDFs/notes with page boundaries.

Document processing and indexing

1. Load PDF/text files.
2. Extract text while preserving page boundaries where possible.
3. Clean extraction artifacts without changing meaning.
4. Chunk documents and assign stable chunk IDs.
5. Store document/page/subject/chunk metadata.
6. Generate embeddings.
7. Build FAISS indexes.
8. Save the index plus a metadata map from vector ID to source chunk.

Chunk size and overlap must be configurable. Starting point: 500–800 tokens with 50–100 token overlap. The final values must be documented and frozen for the main experiment.

Embedding models

- English embedding model for B/C.
- Multilingual embedding model for D.
- Record exact model name/version/revision, dimensions and normalization settings.
- Keep the embedding choice configurable.

Generation model

- Use one generation model consistently across B/C/D where practical.
- Lightweight/quantized local inference or an API model is acceptable.
- Record exact model/version, local/API mode, temperature, max_new_tokens and relevant settings.
- Keep generation temperature fixed for the main benchmark.
- Do not claim local execution if an API was used.

Translation component

- C requires Hindi→English translation before retrieval.
- Final answer must be available in Hindi; use a declared English→Hindi step if needed.
- Record exact translation model/service.
- Save original Hindi question, translated English query and final Hindi answer separately.
- Record failures/timeouts instead of silently replacing failed outputs.

Evaluation benchmark

Create approximately 120 underlying academic information needs. Each should have semantically equivalent English and Hindi questions.

Subject	Suggested count
Algorithms	24
Automata	24
DBMS	24
Operating Systems	24
Computer Networks	24

Recommended question types: Factual/definition, Conceptual explanation, Comparison, Procedure/algorithmic, Multi-step reasoning, Multi-document, Unanswerable. For answerable questions, annotate at least one gold source document and preferably gold page/chunk. For multi-document questions, annotate all required sources.

Benchmark CSV/JSON schema

Field	Required
question_id	Yes
subject	Yes
category	Yes
question_en	Yes
question_hi	Yes
gold_answer	Yes
answerable	Yes
gold_document_id	Yes for answerable
gold_page	Where available
gold_chunk_id	Preferred
notes	Optional

Required per-query experiment log

Field	Purpose
run_id	Unique run identifier
timestamp	Execution time
question_id	Benchmark link
system_id	A/B/C/D
original_question	Exact input
translated_question	C only
retrieved_chunks	Ordered chunk IDs
retrieval_scores	Ranking scores/distances
source_documents	Document/page/source metadata
generated_answer	Exact final answer
intermediate_answer_en	Useful for C
model_name	Generation model
embedding_model	Embedding model
translation_model	C translation model/service
top_k	Retrieval depth
temperature	Generation setting
retrieval_latency_ms	Retrieval time
translation_latency_ms	Translation time
generation_latency_ms	Generation time
total_latency_ms	End-to-end time
error_status	Success/failure
error_message	Failure details

Metrics:

1. Retrieval

- Recall@1
- Recall@3
- Recall@5
- MRR
- Optional nDCG@5

Define exactly what counts as a relevant/gold retrieval before the final run.

2. Answer quality

- Correctness
- Faithfulness/groundedness
- Answer relevance
- Completeness

3. Cross-lingual

- Translation adequacy/semantic preservation
- Technical terminology preservation
- English–Hindi answer semantic consistency

4. Efficiency

- Retrieval latency
- Translation latency
- Generation latency
- Total latency
- Peak RAM/VRAM where practical
- Model/resource footprint

Error taxonomy

Code	Error type	Definition
E1	Translation drift	Translation changes/loses the intended technical meaning.
E2	Retrieval failure	Relevant source/chunk is absent or ranked too low.
E3	Generation failure	Evidence is adequate but the LLM answer is incorrect/incomplete.
E4	Language/terminology failure	Substance is correct but Hindi wording/technical terminology is poor.

Minimal Streamlit UI : **Do not spend major effort on visual design. Logging and reproducibility are more important.**

Project structure

```
cross_lingual_rag/
├── app.py
├── config.yaml
├── requirements.txt
├── README.md
├── data/
│   ├── corpus/
│   ├── benchmark/
│   └── processed/
├── indexes/
│   ├── english/
│   └── multilingual/
├── src/
│   ├── ingestion.py
│   ├── chunking.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── translation.py
│   ├── generation.py
│   ├── retrieval.py
│   ├── pipelines.py
│   ├── evaluation.py
│   ├── logging_utils.py
│   └── utils.py
├── prompts/
```

```
│   └── rag_prompt.txt
├── results/
│   ├── raw/
│   ├── csv/
│   └── summaries/
└── tests/
    └── test_basic_pipeline.py
```

Configuration file

```
embedding:
  english_model: "<exact model>"
  multilingual_model: "<exact model>"
  normalize_embeddings: true

retrieval:
  top_k: 5
  chunk_size: 700
  chunk_overlap: 80
  metric: "cosine"

generation:
  model: "<exact model>"
  temperature: 0
  max_new_tokens: 512

translation:
  provider: "<local model/API>"
  model: "<exact model/service>"

evaluation:
  benchmark_file: "data/benchmark/questions.csv"

systems:
  A_direct_llm: true
  B_english_rag: true
  C_translation_rag: true
  D_multilingual_rag: true
```

Fixed generation prompt

```
You are an academic question-answering assistant.
Use only the supplied retrieved academic context to answer the question.
If the context is insufficient, say so rather than inventing an answer.
Question: {question}
Retrieved academic context:{context}
Give a concise, technically accurate answer.
```

Experimental controls

- Same corpus for B, C and D.
- Same underlying paired English/Hindi information needs.
- Same top-k in the main experiment.
- Same generation model/settings where practical.
- Same chunking/index construction for comparable English retrieval.
- No manual editing of answers.
- No prompt changes per question in the main run.
- Run the benchmark automatically rather than manually.

Reproducibility

- requirements.txt with versions
- README with setup and execution commands
- config.yaml
- benchmark file
- corpus metadata
- saved indexes or reproducible index-building procedure
- raw results
- evaluation scripts
- model identifiers
- prompt version
- seed where applicable

Development sequence

- Set up repository, environment and configuration.
- Implement PDF ingestion and page-aware chunking.
- Implement English embeddings + FAISS.
- Implement multilingual embeddings + FAISS.
- Implement common retrieval interface.
- Implement generation interface.
- Implement translation interface.
- Implement A/B/C/D pipelines.
- Implement complete logging.
- Run a 10-question smoke test.
- Finalize the ~120-question paired benchmark.
- Run automated retrieval evaluation.
- Export answers for quality evaluation.
- Freeze configuration and run the final benchmark.
- Deliver code, raw outputs, summaries and README.

Research outputs the software must enable

- Recall@1/3/5 and MRR comparison for B/C/D.
- English vs Hindi comparison.
- Correctness/faithfulness/relevance/completeness comparison.
- Translation-drift analysis for C.
- E1–E4 error distribution.
- Latency comparison.
- Cases where C succeeds and D fails, and vice versa.
- Cases where retrieval fails.
- Cases where retrieval is correct but generation fails.